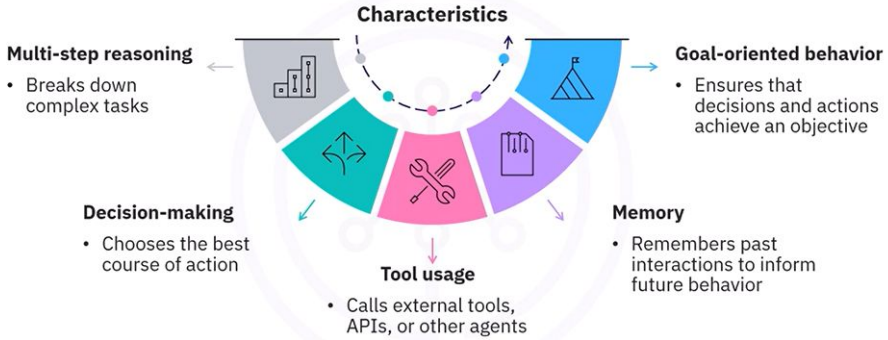
Agentic AI with LangGraph, CrewAI, AutoGen and BeeAI

What is an AI agent?

AI agents

- Systems that operate autonomously
- Make decisions and take actions
- Accomplish goals
- Serve as proactive problem-solvers
- Navigate complex environments

Key Characteristics of AI agents



Traditional AI versus agentic AI

Calculator versus research assistant

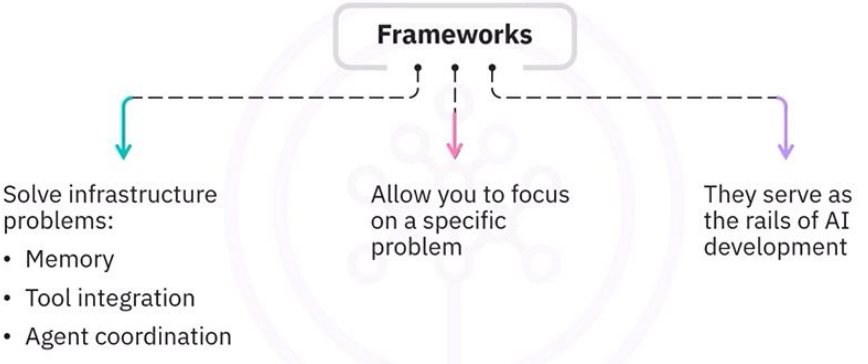
Calculator

- Responds to inputs

Research assistant

- Understands your goal
- Breaks it into steps
- Uses tools to gather information
- Keeps working until the problem is solved

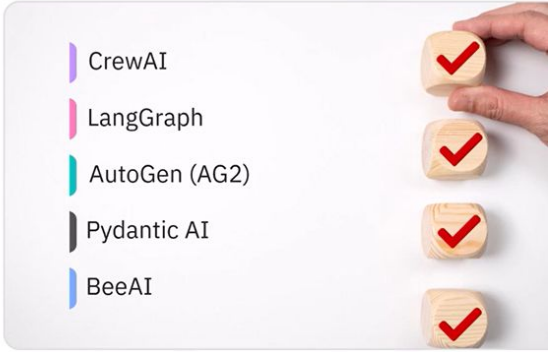
Why use frameworks



Building agentic systems with frameworks

Without Frameworks:	With Frameworks:
Complex message passing protocols	Built-in communication protocols
Manual state synchronization	Automatic state management
Custom coordination logic	Predefined coordination patterns
Error handling across agents	Distributed error handling
Monitoring and debugging challenges	Integrated monitoring tools

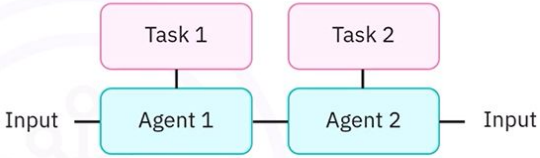
A glimpse at the frameworks



Get started with CrewAI

CrewAI

- Simulates multi-agent collaboration
- Helps define agents with distinct roles and tasks
- Helps accomplish complex goals
- Makes it easy for agents to collaborate
- Simplifies collaboration among agents

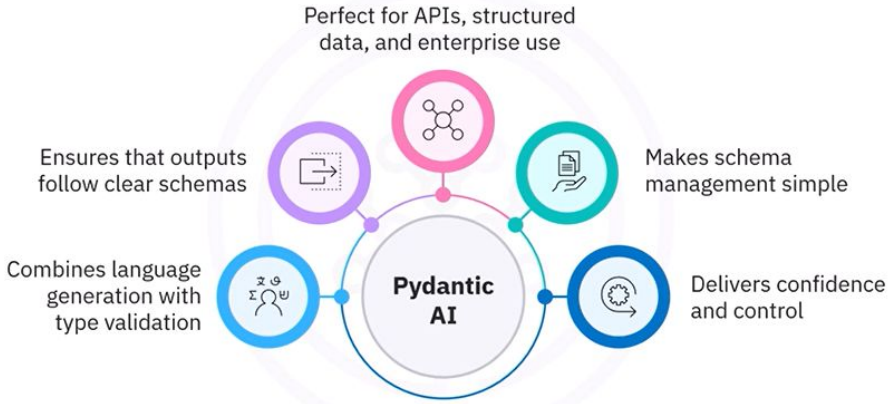


When to use LangGraph

LangGraph

- Great for building complex workflows
- Integrates LLMs, tools, and LangChain
- Use cases
  - Multi-step processes
  - Document workflows
  - Decision trees

Get started with Pydantic AI



Framework comparison

CrewAI and AutoGen

- Offer the smoothest onboarding
- Ideal for rapid prototyping, education, or lightweight systems

LangGraph and Pydantic AI

- Offer greater control
- LangGraph: Suits structured workflows, state management, and error recovery
- Pydantic AI: Ensures reliable, type-safe outputs

BeeAI

- Focuses on enterprise-grade deployment
- Built for scalability and operational robustness

Real-world applications

CrewAI

Powers content pipelines and automated reporting

LangGraph

Handles document and customer service workflows

AutoGen

Supports educational platforms and technical support systems

Pydantic AI

Ensures reliable API services and data validation systems

BeeAI

Manages enterprise automation

Recap

- Agentic AI agents are autonomous systems that make decisions and take action to achieve goals
- Multi-agent systems use specialized agents to work in parallel, offering benefits such as scalability, modularity, and fault tolerance
- CrewAI helps simulate team-style collaboration by assigning clear roles to agents
- LangGraph lets you build structured workflows using nodes and edges

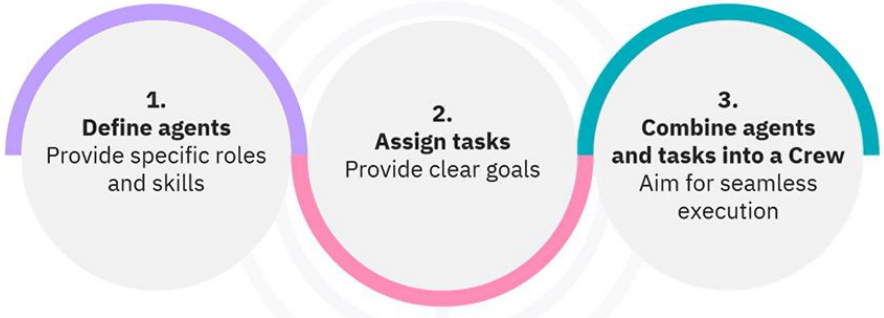
Recap

- AutoGen supports agent-to-agent and agent-to-human conversations
- Pydantic AI ensures that outputs match defined schemas, reducing ambiguity—ideal for API interactions and production environments
- BeeAI is built for enterprise deployment, offering tool integration, memory management, and scalable agent coordination

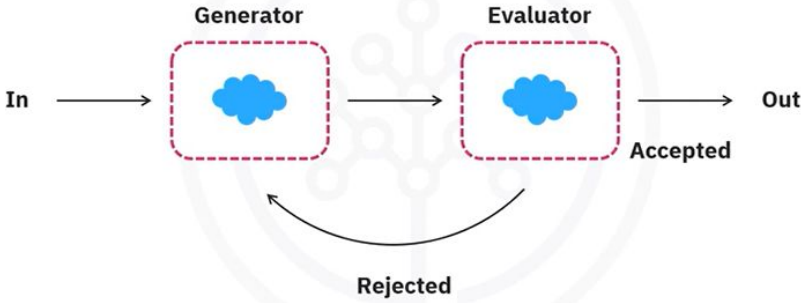
What you will learn

- Explain how agentic frameworks like CrewAI, LangGraph, AutoGen, and BeeAI structure multi-agent workflows
- Describe how agents are created, assigned tasks, and coordinated within each framework
- Interpret code snippets that define workflows, agents, and control logic
- Compare common use cases and limitations across the four frameworks

CrewAI design process outline



Reflection or evaluator-optimizer



Get started

Role

- Import essential libraries

```
import os
from crewai import Agent, Task, Crew, Process, LLM
from crewai_tools import SerperDevTool

# Setup
os.environ['SERPER_API_KEY'] = "your_api_key_here"
search_tool = SerperDevTool()

llm = LLM(
    model="watsonx/meta-llama/llama-3-3-70b-instruct",
    base_url="https://us-south.ml.cloud.ibm.com",
```

Create the agents

- Specify the role, goal, and backstory

```
# Agents
research_agent = Agent(
    role='Senior Research Analyst',
    goal='Uncover cutting-edge information and insights on any subject',
    backstory="Expert researcher with extensive experience in gathering and analyzing information",
    llm=llm,
    tools=[search_tool]
)
writer_agent = Agent(
```

Create tasks

- Specify parameters

```
# Tasks
research_task = Task(
    description="Analyze the major {topic}, identifying key trends and technologies",
    agent=research_agent,
    expected_output="A detailed report on {topic} including trends and impact"
)

writer_task = Task(
    description="Create an engaging blog post based on research findings about {topic}",
    agent=writer_agent,
```

Create a Crew object

- Execute the pipeline

```
# Crew
crew = Crew(
    agents=[research_agent, writer_agent],
    tasks=[research_task, writer_task],
    process=Process.sequential
)
# Run
topic = "Latest Generative AI breakthroughs"
result = crew.kickoff(inputs={"topic": topic})
```

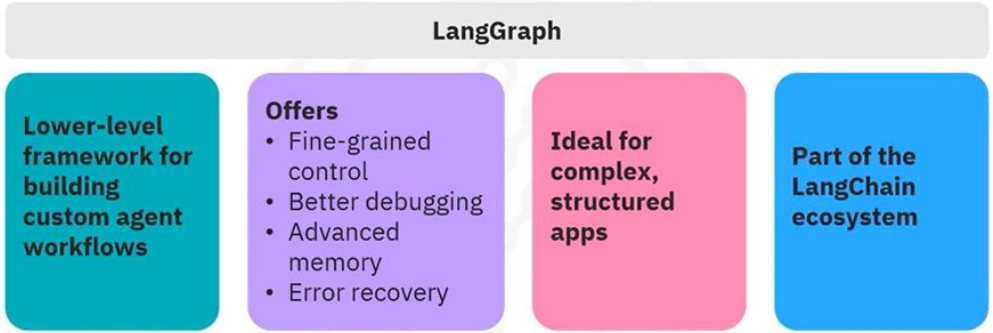
CrewAI use cases

Role-based team simulations
Agents assigned specific roles, tools, and goals

Collaborative task execution
Useful for tasks that require structured collaboration

Drawback
Limited flexibility and difficulty in debugging

LangGraph overview



Import libraries and define state

- Set up the environment

```
from typing import Literal
from pydantic import BaseModel, Field
from langgraph.graph import StateGraph, START, END
from typing_extensions import TypedDict
from IPython.display import Image, display

# State management for the workflow
class State(TypedDict):
    topic: str          # Input topic
    content: str        # Generated content
    feedback: str       # Improvement feedback
```

Structured evaluation scheme

- Define a schema

```
# Structured evaluation schema
class Feedback(BaseModel):
    grade: Literal["good", "needs_improvement"] = Field(
        description="Quality assessment of generated content"
    )
    feedback: str = Field(
        description="Specific suggestions for improvement if needed"
    )
```

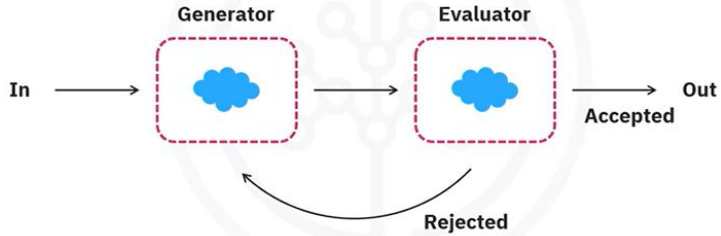
Define the generator and evaluator

- LangGraph gives direct access

```
# Content generator node
def generate_content(state: State):
    """Generate initial content or improve based on feedback"""
    if state.get("feedback"):
        # Incorporate previous feedback
        prompt = f"Create content about {state['topic']} addressing:
{state['feedback']}"
    else:
        # First generation
        prompt = f"Create high-quality content about {state['topic']}"
```

Reflector or evaluator-optimizer

- Routers are conditional nodes



The router function

- Allows you to support graph structures

```
# Routing logic for feedback loop
def route_content(state: State):
    """Decide whether to continue improving or accept content"""
    if state["quality_grade"] == "good":
        return "ACCEPTED"  # Content meets quality standards
    else:
        return "IMPROVE"   # Needs another iteration
```

Build the graph

- Add nodes and define their edges

```
# Add processing nodes
workflow.add_node("generate_content", generate_content)
workflow.add_node("evaluate_content", evaluate_content)

# Define workflow edges
workflow.add_edge(START, "generate_content")
workflow.add_edge("generate_content", "evaluate_content")

# Conditional routing based on evaluation
workflow.add_conditional_edges(
    "evaluate_content",
    route_content,
```

LangGraph use cases

Complex workflow automation

Ideal for scenarios requiring advanced memory and structured workflows

Multi-agent system management

Suitable for applications needing precise control over interactions between multiple agents

AutoGen/AG2 overview

Approach: Dialogue-driven multi-agent conversations

Strengths: Easy prototyping, intuitive chat interface, code execution

Best for: Conversational agents, research assistants

Note: Originally developed by Microsoft

Build a multi-agent study assistant

- Build a simple yet powerful study assistant
- Features three agents:
  1. **student_agent** (understands the query)
  2. **concept_analysis_agent** (analyzes the query)
  3. **study_tip_agent** (offers study tips based on the query)

Define roles for study agents

```
student_agent = ConversableAgent(
    name="student",
    system_message="You ask for help on a topic you're studying.",
    llm_config=llm_config
)

concept_analysis_agent = ConversableAgent(
    name="concept_analysis",
    system_message="You analyze and explain the main concepts of the topic. Do not provide study tips.",
    llm_config=llm_config
```

Coordinate agents with GroupChatManager

- GroupChatManager facilitates turn-based collaboration

```
groupchat = GroupChat(
agents=[concept_analysis_agent, study_tip_agent],
messages=[],
max_round=3,
speaker_selection_method="round_robin" )

manager = GroupChatManager(name="manager", groupchat=groupchat)
```

Launch the study assistant chat

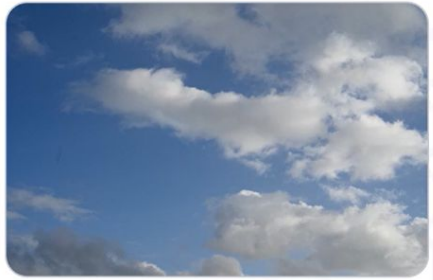
- Conversation flows through other agents

```
def start_study_assistant():
    print("\nWelcome to the AI Study Assistant!")
    topic = input("What topic would you like help with?")

    print("\nAnalyzing topic...")
    response = student_agent.initiate_chat(
        manager,
        message=f"I'm studying {topic}. Can you help me understand and remember it?"
    )

    if not response:
        response = study_tip_agent.initiate_chat(
```

BeeAI use case



BeeAI framework

- Enables modular agent workflows with integrated tools

Example: Three-agent collaboration to generate a location report

- **Researcher**: Uses Wikipedia for historical context
- **WeatherForecaster**: Uses OpenMeteo for live weather
- **DataSynthesizer**: Fuses historical and weather data into a coherent summary

Setup and initialization in BeeAI

- Define the workflow

```
pip install beeai-framework

from beeai_framework.backend.chat import ChatModel
from beeai_framework.tools.search.wikipedia import WikipediaTool
from beeai_framework.tools.weather.openmeteo import OpenMeteoTool
from beeai_framework.workflows.agent import AgentWorkflow, AgentWorkflowInput

llm = ChatModel.from_name("ollama:granite3.3:8b")
workflow = AgentWorkflow(name="Smart assistant")
```

Running the workflow

- Final output is a fused summary

```
        AgentWorkflowInput(prompt="Provide a comprehensive weather summary for
Saint-Tropez today."),

        AgentWorkflowInput(prompt="Summarize the historical and weather data for
Saint-Tropez."),]
).on(
    "success",
    lambda data, event: print(
        f"\n-> Step '{data.step}' has been completed
with:\n{data.state.final_answer}"
    )
)
```

Introduction

Focus:
Text input and outputs using LLMs

Extendable:
Can be modified to use agents and tool-based output

Sequential design pattern

Simplest pattern

Pass one LLM's output directly into the next

Task specialization

Agents handle specialized tasks better

Task breakdown

Complex tasks are broken into manageable steps

Prompt chaining

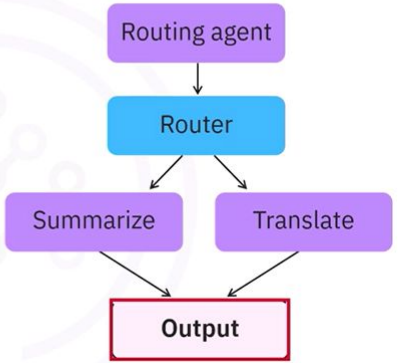
Job description → Resume summary → Write cover letter → Cover letter

Example: Writing a cover letter

1. A technical expert drafts the resume summary
2. A professional writer crafts the letter

Routing design pattern

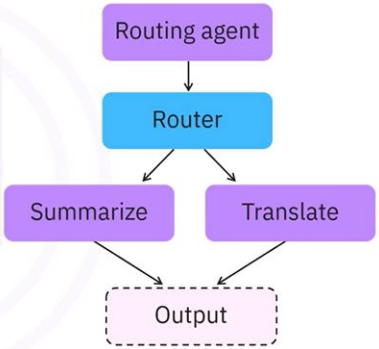
- Build a system that intelligently selects the right agent for a task
- Create a routing agent that analyzes the query
- Decide whether to summarize or translate the input
- Direct the flow to the appropriate agent node
- The end node is labeled as the output for clarity



Routing design pattern

- Set router_node as the entry point

```
workflow = StateGraph(RouterState)
workflow.add_node("router_node", router_node)
workflow.add_node("summarize", summarize_node)
workflow.add_node("translate", translate_node)
workflow.set_entry_point("router_node")
```



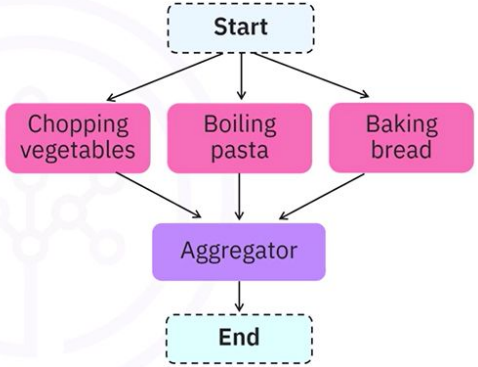
Parallelization pattern

Multiple chefs

- Chop vegetables, boil pasta, and back bread
- Combines everything together

AI workflows

- Splits problems into independent parts



Parallelization pattern

- Add three translation nodes

```
graph = StateGraph(State)
graph.add_node("translate_french", translate_french)
graph.add_node("translate_spanish", translate_spanish)
graph.add_node("translate_japanese", translate_japanese)
graph.add_node("aggregator", aggregator)
```

Recap

- The sequential, or prompt chaining, pattern passes one LLM's output as the next LLM's input, breaking complex tasks into manageable steps
- The routing pattern uses a router agent to analyze input, determine the task type, and direct execution to the correct agent node
- The parallelization pattern runs multiple independent LLM tasks simultaneously and merges their outputs with an aggregator node
- LangGraph implements each pattern using state variables, nodes, and directed edges, allowing structured workflows with clear execution flow and outputs

Understanding the orchestrator design pattern

Scenario: Party planner for multi-themed dinners

- Guest requests vary daily
- Difficult to make menus

Core problem

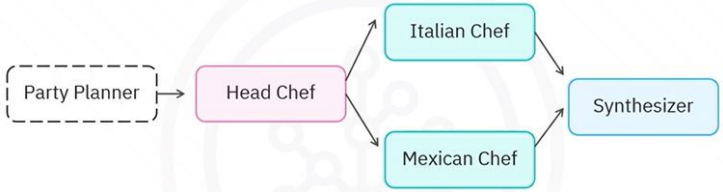
- Earlier, workflows were static
- What if you don't know a task's complexity?

The case for orchestrator pattern

- Lets the planner coordinate tasks dynamically
- Adapts in real time to each event's needs

Task assignment and parallel execution

"Please prepare Italian pasta and Mexican tacos for tonight's themed dinner"



Party planner gets a request

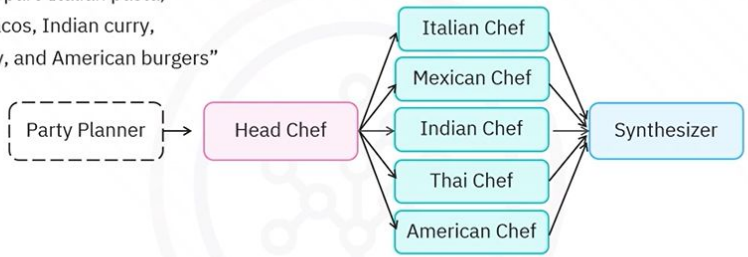
Orchestrator agent analyzes the request

Italian and Mexican chef agents work in parallel

Synthesizer combines their outputs

Task assignment and parallel execution

"Please prepare Italian pasta,
Mexican tacos, Indian curry,
Thai stir-fry, and American burgers"

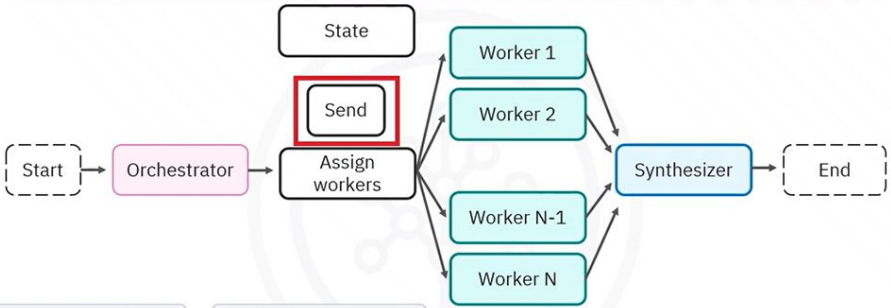


The request grows to include multiple dishes

Five chefs are recruited

Contributions are merged into a buffet plan

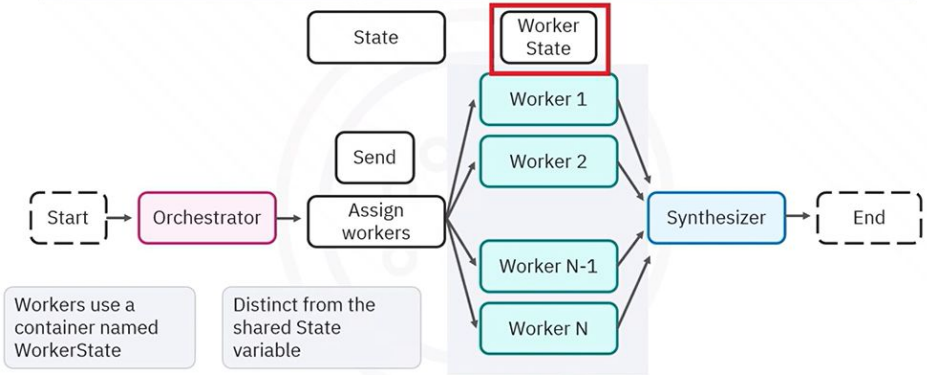
Assign workers node



"assign workers" node decides routes

Send() function sends structured data

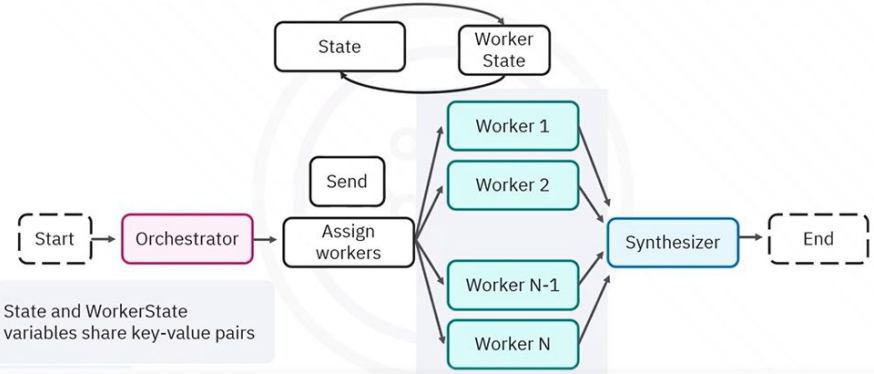
WorkerState explained



Workers use a container named WorkerState

Distinct from the shared State variable

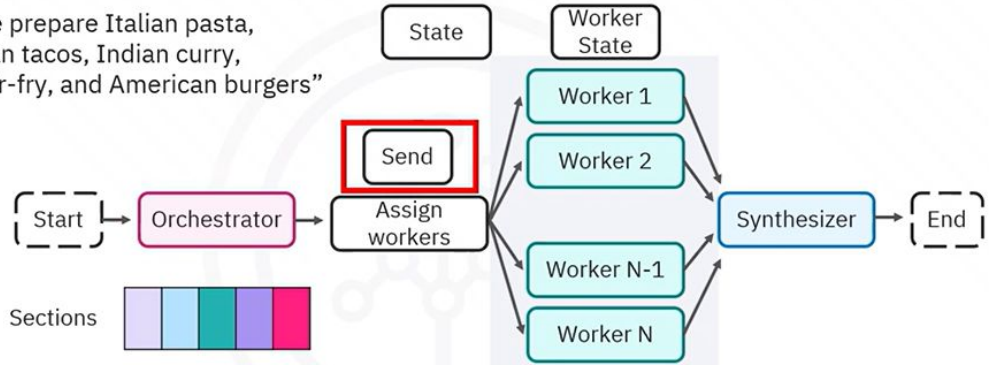
State and WorkerState variables



State and WorkerState variables share key-value pairs

How the orchestrator breaks down tasks

"Please prepare Italian pasta, Mexican tacos, Indian curry, Thai stir-fry, and American burgers"



Planner provides input

Orchestrator outputs sections

send() passes each section to a worker

State variables overview

```
class State(TypedDict):
    "meals": str
    "sections": List[Dish]
    completed_menu: Annotated[List[str], operator.add]
    final_meal_guide: str
```

These variables appear in the table

Variable Name	Value
meals	
sections	
completed_menu	
final_meal_guide	

WorkerState structure and task passing

```
class State(TypedDict):
    meals: str
    sections: List[Dish]
    completed_menu: Annotated[List[str], operator.add]
    final_meal_guide: str


class WorkerState(TypedDict):
    section: Dish
    completed_menu: Annotated[list, operator.add]
```

Includes the complete menu list

Variable Name	Value
section	
completed_menu	

Building the Orchestrator LLM

Create the LLM for the orchestrator

Chain the prompt to the LLM with planner_pipe Please check.

```
        "The user wants to prepare the following meals: {meals}\n\n"
        "For each meal, return a section with:\n"
        "- the name of the dish\n"
        "- a comma-separated list of ingredients needed for that dish.\n"
        "- the cuisine or cultural origin of the food"
    )
])

planner_pipe = dish_prompt | llm.with_structured_output(Dishes)
```

Orchestrator input and output flow

Takes the meal request from the state

Passes it through the planner_pipe

Returns a structured dish list

```
def orchestrator(state: State):
    """Orchestrator that generates a structured dish list from the given meals."""

    # use the planner_pipe LLM to break the user's meal list into structured dish sections
    dish_descriptions = planner_pipe.invoke({"meals": state["meals"]})

    # return the list of dish sections to be passed to worker nodes
    return {"sections": dish_descriptions.sections}
```

→ Orchestrator →

Extracting and routing tasks

Accesses the State variable

Extracts the sections key

Retrieves the Dish objects

```
def assign_workers(state: State):
    return [Send("chef_worker", {"section": s}) for s in state["sections"]]
```

Variable Name	Value
meals	
sections	
completed_menu	
final_meal_guide	

Item Name	Location Cuisine	Ingredients
Pasta	Italian	spaghetti, olive oil, garlic, tomato sauce..
Tacos	Mexican	taco shells, ground beef, taco seasoning, lettuce..
Curry	Indian	chicken or tofu, curry powder, coconut milk
Stir-Fry	Thai	noodles, soy sauce, vegetable oil, bell peppers
Burgers	American	ground beef, burger buns, cheddar..

From sections to worker nodes

Each variable represents a Dish in the sections list

Send() passes each dish to chef_worker

```
def assign_workers(state: State):
    return [Send("chef_worker", {"section": s}) for s in state["sections"]]
```

Item Name	Location Cuisine	Ingredients
Pasta	Italian	spaghetti, olive oil, garlic, tomato sauce..
Tacos	Mexican	taco shells, ground beef, taco seasoning, lettuce..
Curry	Indian	chicken or tofu, curry powder, coconut milk
Stir-Fry	Thai	noodles, soy sauce, vegetable oil, bell peppers
Burgers	American	ground beef, burger buns, cheddar..

WorkerState

Variable Name	Value
meals	
sections	
completed_menu	
final_meal_guide	

Worker processing with chef_pipe

Creates a completed menu by receiving a Dish object

Extracts the name, location, and ingredients

Returns the meal plan in the completed_menu list

```
}
return {"completed_menu": [meal_plan.content]}
```

Variable Name	Value
meals	
sections	
completed_menu	
final_meal_guide	

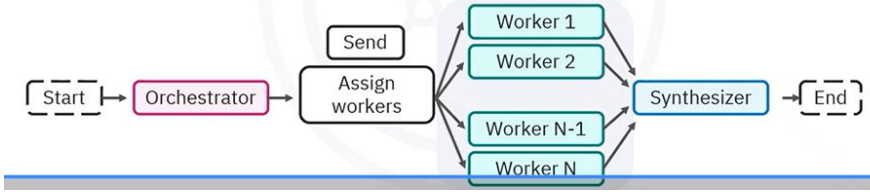
Item Name	Location Cuisine	Ingredients
Pasta	Italian	spaghetti, olive oil, garlic, tomato sauce..
Tacos	Mexican	taco shells, ground beef, taco seasoning, lettuce..
Curry	Indian	chicken or tofu, curry powder, coconut milk
Stir-Fry	Thai	noodles, soy sauce, vegetable oil, bell peppers
Burgers	American	ground beef, burger buns, cheddar..

Invoking the graph for meal preparation

Add edges from worker nodes

Compile the graph

```
orchestrator_worker_builder.add_edge("chef_worker", "synthesizer")
orchestrator_worker_builder.add_edge(START, "orchestrator")
orchestrator_worker_builder.add_edge("synthesizer", END)
orchestrator_worker = orchestrator_worker_builder.compile()
```



Recap

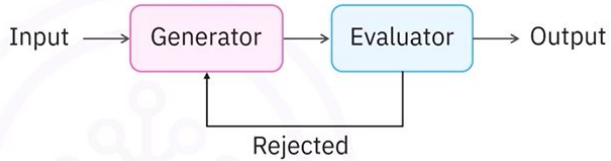
- The orchestrator pattern manages dynamic workflows by breaking down complex requests and assigning tasks to specialized worker agents
- WorkerState and shared State variables keep task-specific details and shared context separate yet accessible for coordinated processing
- The assign workers node routes tasks in parallel, and each worker generates detailed outputs based on structured inputs
- A synthesizer node combines all worker outputs into a unified result, enabling flexible and scalable workflow execution

The evaluator-optimizer pattern

A query is passed to the generator LLM

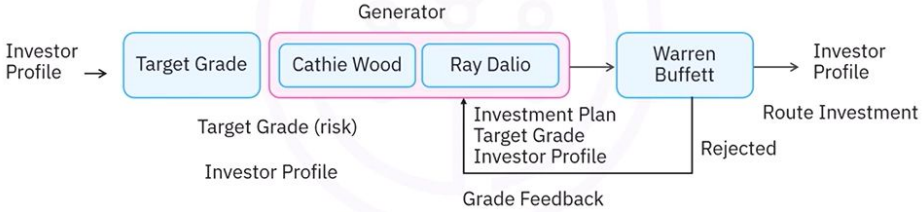
Output goes to the evaluator LLM

Process repeats until the result meets the target criteria



Workflow graph

- LLM suggests a target risk grade
- Profile and grade are passed to the generator
- Cathie Wood produces an investment plan for Warren Buffett's evaluator
- Warren gives a risk grade and feedback
- Plan goes to Ray Dalio's LLM for refinement
- Plan is iteratively refined
- Profile is finalized if the grade matches the target



Recap

- The evaluator–optimizer pattern refines LLM outputs through iteration until they meet target criteria
- State variables track investor profiles, target risk grades, evaluator feedback, and iteration counts
- Generator and evaluator personas—Cathie Wood, Ray Dalio, and Warren Buffett—drive strategy creation, assessment, and refinement
- LangGraph connects grading, generation, evaluation, and routing nodes into a reflection loop that assembles the full investment planning workflow

What you will learn

Define the core components of CrewAI

Describe how agents and tasks work together

Identify how CrewAI handles sequential task execution

Recognize how to access the result and evaluate usage metrics using the CrewOutput object

An introduction to CrewAI

- Designed for multi-agent collaboration
- Features four components
  - Task
  - Agent
  - Tool
  - Flow

Task

Like a director

Has a clear vision

Defines two key parameters
- Goal
- Agent

Note: In CrewAI, you typically define the agent first.

Task

Description
- It's what you want your AI agent to do
- Director analogy: "Make an eye-catching commercial that will sell our car."

Task

Expected output

- It defines specific requirements
  - Example: “Make it a 30-second car commercial.”
- May include variable parameters
  - Example: “Make a 20-second commercial about boats.”

Agent

Agent

- Powered by an LLM
- Defined with structured prompts
- Provides more personality

Actor analogy

- Have skills and a role
- Need direction from the director
- Example: “Drive a car and make it look good”

Agent – Structured prompts

Agent

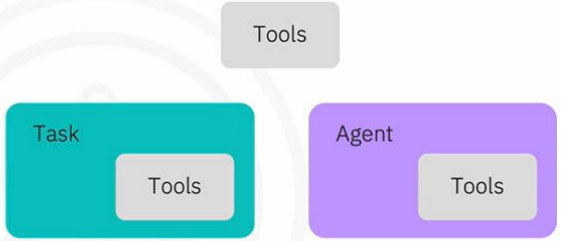
Role
- Defines what kind of expert the agent is
- Example: “Play a cool guy driving a car.”

Goal
- Is the objective guiding the agent’s decisions
- Example: “Drive the car smoothly and look confident.”

Backstory
- Provides context and behavior
- Example: “You are a successful entrepreneur who drives expensive cars and exudes confidence.”

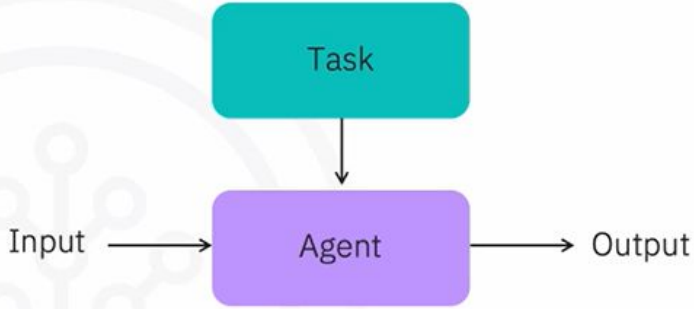
Tools

- Standard components used in AI workflows
- Can be used by either the Agent or the Task
- Actor: A car or stylish clothing
- Director: The camera or the microphone



Putting it together

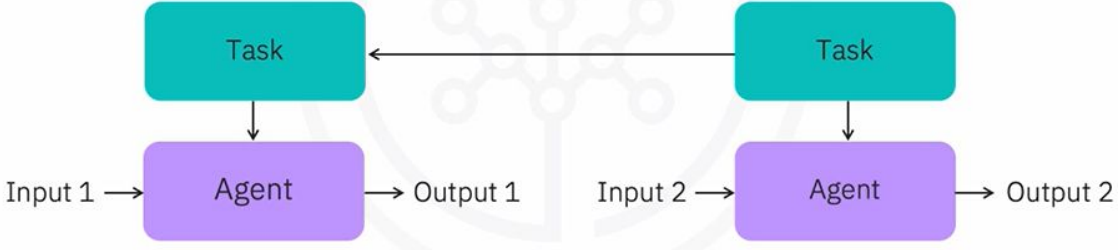
- Create an Agent object
- Define a task
- Input flows into the agent



Flows

Flows

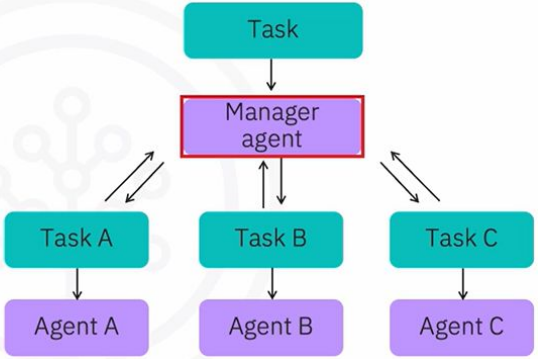
- Define how tasks run and how agents interact
- Are set via a parameter in the main Crew object
- One type is sequential
- Output is the input for the next



Flows

Hierarchical

- A manager agent assigns and oversees tasks
- Best used when autonomy and flexibility are needed



CrewAI in Python

CrewAI content pipeline

- Automated system with two AI agents
  - Research Analyst
  - Content Strategist

Initialize the LLM

- Pass the LLM to each agent

```
rom crewai import LLM

lm = LLM(

      model="watsonx/meta-llama/llama-3-3-70b-instru
      base_url="https://us-south.ml.cloud.ibm.com",
      project_id="skills-network",
```

Define the research

- Set verbose=True for debugging

```
from crewai_tools import SerperDevTool
from crewai import Agent

research_agent = Agent(

  role='Senior Research Analyst',

  goal='Uncover cutting-edge information and insights on any subject with
comprehensive
analysis',

  backstory="""You are an expert researcher with extensive experience in gathering,
analyzing, and synthesizing information across multiple domains.

  Your analytical skills allow you to quickly identify key trends, separate fact
from opinion, and produce insightful reports on any topic.
```

Define the research

- Set allow_delegation=False

```
  backstory="""You are an expert researcher with extensive experience in gathering,
analyzing, and synthesizing information across multiple domains.

  Your analytical skills allow you to quickly identify key trends, separate fact
from opinion, and produce insightful reports on any topic.

  You excel at finding reliable sources and extracting valuable information
efficiently.""",

  verbose=True,

  allow_delegation=False,

  llm = llm,

  tools=[SerperDevTool()]
```

Assign research tasks

- "topic" will be the input variable

```
from crewai import Task

research_task = Task(

  description="Analyze the major {topic}, identifying key trends and technologies.
  Provide a detailed report on their potential impact.",

  agent=research_agent,

  expected_output="A detailed report on {topic}, including trends, emerging
  technologies, and their impact."

)
```

Define the writer agent

- Goal: Craft well-structured and engaging content based on research findings

```
from crewai import Agent

writer_agent = Agent(

  role='Tech Content Strategist',

  goal='Craft well-structured and engaging content based on research findings',

  backstory="""You are a skilled content strategist known for translating
  complex topics into clear and compelling narratives. Your writing makes
  information accessible and engaging for a wide audience.""",

  verbose=True,

  llm = llm,
```

Assign the writer task

- Use the research findings from previous step

```
from crewai import Task

writer_task = Task(

  description="Create an engaging blog post based on the research findings about
{topic}. Tailor the content for a tech-savvy audience, ensuring clarity and
interest.",

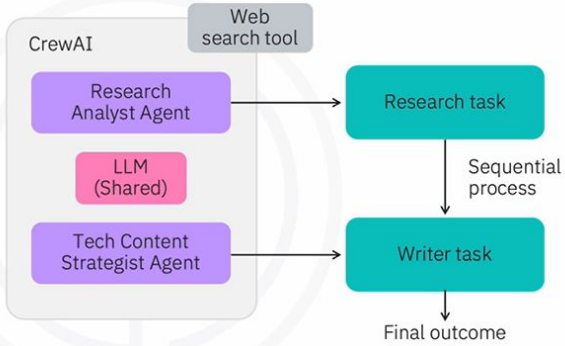
  agent=writer_agent,

  expected_output="A 4-paragraph blog post on {topic}, written clearly and
engagingly for tech enthusiasts."

)
```

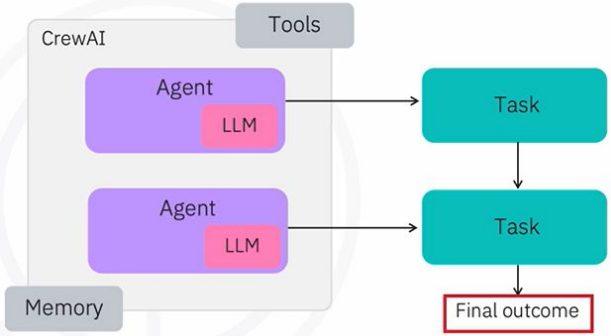
Crew object

- Build a two-agent Crew
- Analyze recent Generative AI breakthroughs
- Turn the research into blog posts
- Crew object creates a coordinated workflow



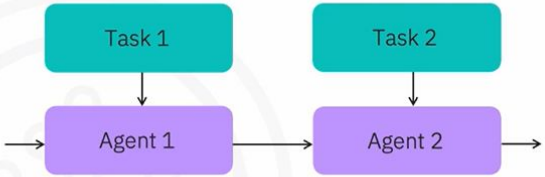
Crew object

- Powered by an LLM
- Assigned tasks that define their role
- Uses tools like search engines and APIs
- Uses memory to maintain context
- Produces the final outcome



Bringing it all together

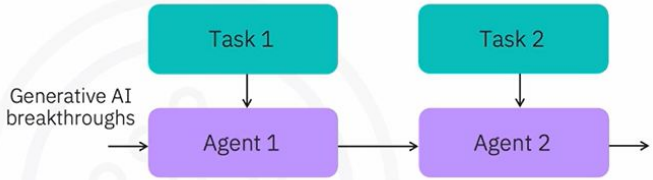
- Assemble the crew
- Create a Crew object
- Pass in the agents and their tasks
- Specify the process type




```
from crewai import Crew, Process
crew = Crew(
    agents=[research_agent, writer_agent],
    tasks=[research_task, writer_task],
    process=Process.sequential,
    verbose=True
)
```

Bringing it all together

- Run the crew using the .kickoff() method
- Passed to the {topic} parameter



A 4-paragraph blog post on {topic}, written clearly and engagingly for tech enthusiasts

```
result = crew.kickoff(inputs={"topic": "Generative AI breakthroughs"})
```

Output structure

- "raw" contains the unified output

```
CrewOutput(
  raw="The final generated output combining all tasks",
  tasks_output=[
    TaskOutput(
      description="What the research task was meant to achieve",
      expected_output="A detailed report on Latest Generative AI breakthroughs",
      summary="Summarized task prompt for analysis",
      raw="Full detailed report generated by the Research Analyst agent",
      agent="Senior Research Analyst"
    ),
    TaskOutput(
      description="Goal of the writing task using research findings",
      expected_output="A blog post tailored for tech-savvy readers"
```

Final output

```
final_output = result.raw
print("Final output:", final_output)
```

```
Final output: The latest generative AI breakthroughs have been making waves in the
tech industry, with numerous advancements and innovations being reported in recent
times. According to the search results, some of the key trends and emerging
technologies in generative AI include improved conversational abilities and
personalization, accelerated drug discovery, enhanced scientific research,
autonomous AI, and multimodal generative AI. These advancements are expected to have
a significant impact on various industries, including healthcare, finance,
education, and entertainment.


One of the most significant applications of generative AI is in the field of
healthcare. For instance, generative AI can be used to develop personalized
medicine, create personalized educational content, and generate realistic videos and
images. Additionally, generative AI can be used to improve customer service, enhance
user experience, and increase efficiency in various business processes. …
```

Recap

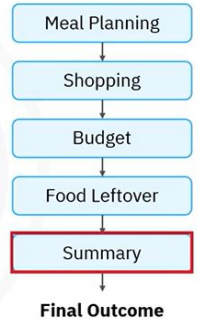
- CrewAI is designed for multi-agent collaboration, with agents assigned clear roles and tasks to simulate human-like teamwork
- A task defines what an agent should do, including a goal, description, and expected output
- An Agent is powered by an LLM and guided by structured prompts: role, goal, and backstory to simulate personality and expertise
- Tools like APIs or search engines enhance agent capabilities and can be assigned to both agents and tasks
- The Crew object ties agents, tasks, tools, and flow together into a coordinated system
- CrewOutput captures the final result, task outputs, and token usage, giving a full snapshot of what was generated and its cost

What you will learn

- Build a multi-agent meal planning system using CrewAI
- Structure agent outputs with Pydantic for clean, reusable data
- Define agents and tasks in YAML for flexible configuration
- Use @CrewBase to load YAML agents and run complete planning workflows

Introduction

- Flexible meal planning system
- Each agent has a clear role:
  - Planning meals
  - Organizing shopping
  - Checking the budget
  - Handling leftovers
  - Combining everything into a final guide



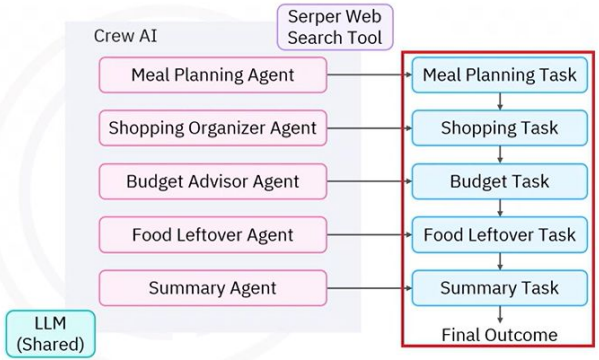
Introduction

Leftovers agent uses YAML and the CrewBase class

Pydantic models ensure each agent's output is structured

Crew overview

- Crew structure has 5 agents
- Powered by a shared LLM
- Uses external tools
- Tasks are linked sequentially



Crew overview

1. Define agents

Provide specific roles and skills

```
from crewai import Agent, Task, Crew, Process

meal_planner_agent = Agent(
    role="",
    goal="",
    backstory="",
    llm=llm
)

meal_planning_task = Task(
```

Crew overview

2. Assign tasks

Provide clear goals

```
meal_planning_task = Task(
    description="",
    expected_output="",
    agent=meal_planning_agent
)

crew = Crew(
    agents=[meal_planner_agent, shopping_organizer_agent, …, summary_agent],
    tasks=[meal_planning_task, shopping_task, …, summary_task]
```

Crew overview

3. Combine agents and tasks into a Crew

Aim for seamless execution

```
    agent=meal_planning_agent
)

crew = Crew(
    agents=[meal_planner_agent, shopping_organizer_agent, …, summary_agent],
    tasks=[meal_planning_task, shopping_task, …, summary_task]
)

result = crew.kickoff()
print(result)
```

Initialize the LLM

The LLM will be passed to each agent

```
from crewai import LLM


llm = LLM(

    model="watsonx/ibm/granite-3-3-8b-instruct",
    base_url="https://us-south.ml.cloud.ibm.com",
    project_id="skills-network",

)
```

Create a grocery shopping structure with Pydantic

- Type hints ensure the data is structured and complete

```
from pydantic import BaseModel, Field

class GroceryItem(BaseModel):
    """Individual grocery item"""

    name: str = Field(description="Name of the grocery item")
    quantity: str = Field(description="Quantity needed (e.g., '2 lbs', '1 gallon')")
    estimated_price: str = Field(description="Estimated price (e.g., '$3-5')")
    category: str = Field(description="Store section (e.g., 'Produce', 'Dairy')")
```

Create a grocery shopping structure with Pydantic

- Features 4 fields

```
class MealPlan(BaseModel):
    """Simple meal plan"""

    meal_name: str = Field(description="Name of the meal")
    difficulty_level: str = Field(description="'Easy', 'Medium', 'Hard'")
    servings: int = Field(description="Number of people it serves")
    researched_ingredients: List[str] = Field(description="Ingredients found
through research")
```

Create a grocery shopping structure with Pydantic

- It includes three fields

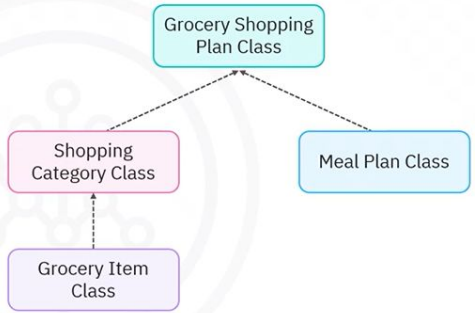
```
from pydantic import BaseModel, Field
from typing import List

class ShoppingCategory(BaseModel):
    """Store section with items"""

    section_name: str = Field(description="Store section (e.g., 'Produce',
'Dairy')")
    items: List[GroceryItem] = Field(description="Items in this section")
    estimated_total: str = Field(description="Estimated cost for this section")
```

Create a grocery shopping structure with Pydantic

- The final class is the GroceryShoppingPlan class
- The arrows represent objects passed into class constructors
- The GroceryShoppingPlan class takes in Meal Plan and Shopping Category objects
- The Shopping Category object contains one or more Grocery Item objects



Create a grocery shopping structure with Pydantic

- Provides a structured view of everything the agents generate

```
class GroceryShoppingPlan(BaseModel):


    """Complete simplified shopping plan"""

    total_budget: str = Field(description="Total planned budget")

    meal_plans: List[MealPlan] = Field(description="Planned meals")

    shopping_sections: List[ShoppingCategory] = Field(description="Organized
by store sections")

    shopping_tips: List[str] = Field(description="Money-saving and efficiency
tips")
```

Create a grocery shopping structure with Pydantic

- Create a GroceryShoppingPlan object
- Pass MealPlan and ShoppingCategory objects


```
weekly_shopping_plan = GroceryShoppingPlan(

    total_budget="$40-50",

    meal_plans=[breakfast_meal, lunch_meal, dinner_meal],

    shopping_sections=[dairy_section, meat_section, pantry_section,
produce_section],

    shopping_tips=[...]

)
```

Define the meal planner agent and task

- Use the Serper Web Search Tool for real-time results

```
from crewai_tools import SerperDevTool

from crewai import Agent, Task, Crew, Process


meal_planner = Agent(

    role="Meal Planner & Recipe Researcher",

    goal="Search for optimal recipes and create detailed meal plans",

    backstory="A skilled meal planner who researches the best recipes online,
considering dietary needs, cooking skill levels, and budget constraints.",

    tools=[SerperDevTool()],

    llm=llm,

    verbose=False
```

Define the meal planner agent and task

- Define the meal planning task

```
meal_planning_task = Task(


    description=(

        "Search for the best '{meal_name}' recipe for {servings} people
within a {budget} budget."

        "Consider dietary restrictions: {dietary_restrictions} and cooking
skill level: {cooking_skill}."

        "Find recipes that match the skill level and provide complete
ingredient lists with quantities."

    ),

    expected_output="A detailed meal plan with researched ingredients,
quantities, and cooking instructions appropriate for the skill level.",
```

Define the meal planner agent and task

- Define the meal planning task

```
        "Consider dietary restrictions: {dietary_restrictions} and cooking
skill level: {cooking_skill}."

        "Find recipes that match the skill level and provide complete
ingredient lists with quantities."

    ),

    expected_output="A detailed meal plan with researched ingredients,
quantities, and cooking instructions appropriate for the skill level.",

    agent=meal_planner,

    output_pydantic=MealPlan,

    output_file="meals.json"

)
```

Define the budget advisor agent and task

- Use the Serper Web Search Tool for price estimates

```
from crewai_tools import SerperDevTool

from crewai import Agent, Task, Crew, Process


budget_advisor = Agent(

    role="Budget Advisor",

    goal="Provide cost estimates and money-saving tips",

    backstory="A budget-conscious shopper who helps families save money on
groceries while respecting dietary needs.",

    tools=[SerperDevTool()],
```

Define the budget advisor agent and task

- budget_task tells the agent to analyze the meal plan and shopping list

```
budget_task = Task(

    description=(

        "Analyze the shopping plan for '{meal_name}' serving {servings}
people."

        "Ensure total cost stays within {budget}. Consider dietary
restrictions: {dietary_restrictions}."

        "Provide practical money-saving tips and alternative ingredients if
needed to meet budget."

    ),
```

Define the budget advisor agent and task

- Outputs markdown to shopping_guide.md

```
        "Provide practical money-saving tips and alternative ingredients if
needed to meet budget."

    ),

    expected_output="A complete shopping guide with detailed prices, budget
analysis, and money-saving tips.",

    agent=budget_advisor,

    context=[meal_planning_task, shopping_task],

    output_file="shopping_guide.md"

)
```

Define the budget advisor agent and task

- Define the leftover agent

```
#agent.yaml
leftover_manager:
  role: >
    Leftover & Waste Reduction Specialist
  goal: >
    Minimize food waste by suggesting creative ways to use leftover
  ingredients and partial quantities
  backstory: >
    A sustainability-focused chef who specializes in zero-waste cooking and
  creative leftover transformations.
  verbose: true
```

Define the budget advisor agent and task

- Define the leftover task

```
 # task.yaml
leftover_task:
  description: >
    Analyze the shopping list for '{meal_name}' and identify ingredients that
  might result in leftovers or partial usage.
    Suggest additional quick meals or recipes that can use these leftover
  ingredients within the {budget} budget.
    Consider dietary restrictions: {dietary_restrictions}.
  expected_output: >
    A list of bonus recipes and creative uses for leftover ingredients,
  helping to maximize the grocery budget and minimize waste.
```

Structure leftover food agent and task with YAML

- Load the leftover agent and task from YAML using the config parameter

```
from crewai import Agent, Task
from crewai.project import CrewBase
from crewai.project.annotations import agent, task

@CrewBase
class LeftoversCrew:
    def __init__(self, llm):
        self.llm = llm

    @agent
    def leftover_manager(self) -> Agent:
```

Structure leftover food agent and task with YAML

- Return valid Agent or Task objects

```
@agent
def leftover_manager(self) -> Agent:
    return Agent(
        config=self.agents_config["leftover_manager"],
        llm=self.llm,
        verbose=True
    )

@task
def leftover_task(self) -> Task:
```

Structure leftover food agent and task with YAML

- CrewBase automatically locates the config folder

```
            llm=self.llm,
            verbose=True
        )

    @task
    def leftover_task(self) -> Task:
        return Task(
            config=self.tasks_config["leftover_task"],
            agent=self.leftover_manager()
        )
```

Instantiate and access leftover food CrewBase class

- Access the YAML-defined agent

```
from leftover import LeftoversCrew

leftovers_cb = LeftoversCrew(llm=llm)

yaml_leftover_manager = leftovers_cb.leftover_manager()

yaml_leftover_task = leftovers_cb.leftover_task()
```

Define the summary agent and task

- Create the Summary Agent

```
from crewai import Agent, Task, Crew, Process


summary_agent = Agent(

    role="Report Compiler",

    goal="Compile comprehensive meal planning reports from all team outputs",

    backstory="A skilled coordinator who organizes information from multiple
specialists into comprehensive, easy-to-follow reports.",
    tools=[],

    llm=llm,
```

Define the summary agent and task

- Use summary_task to gather outputs from all agents

```
summary_task = Task(

    description=(

        "Compile a comprehensive meal planning report that includes:\n"

        "1. The complete recipe and cooking instructions from the meal
planner\n"

        "2. The organized shopping list with prices from the shopping
organizer\n"

        "3. The budget analysis and money-saving tips from the budget
advisor\n"
```

Define the summary agent and task

- Generate a complete report

```
        "4. The leftover management suggestions from the waste reduction
specialist\n"

        "Format this as a complete, user-friendly meal planning guide."

    ),

    expected_output="A comprehensive meal planning guide that combines all
team outputs into one cohesive report.",

    agent=summary_agent,

    context=[meal_planning_task, shopping_task, budget_task,
yaml_leftover_task],
```

Define the summary agent and task

- Create complete_grocery_crew

```
complete_grocery_crew = Crew(
    agents=[meal_planner, shopping_organizer, budget_advisor,
yaml_leftover_manager, summary_agent],

    tasks=[meal_planning_task, shopping_task, budget_task,
yaml_leftover_task, summary_task],

    process=Process.sequential,

    verbose=True
)


# Run the complete crew
```

Define the summary agent and task

- Call .kickoff() with user input to start the crew

```
# Run the complete crew

complete_result = complete_grocery_crew.kickoff(

    inputs={

        "meal_name": "Chicken Stir Fry",

        "servings": 4,

        "budget": "$25",

        "dietary_restrictions": ["no nuts", "low sodium"],

        "cooking_skill": "beginner"
```

Recap

- CrewAI lets you build multi-agent workflows by defining agents with specific roles, goals, and tasks, then grouping them in a Crew for sequential execution
- A shared LLM, like Granite on WatsonX, powers all agents, ensuring consistent reasoning across the workflow
- Pydantic models like GroceryItem, MealPlan, and GroceryShoppingPlan provide clean, validated data structures that agents can pass to one another reliably
- The Serper Web Search tool enables real-time recipe and pricing data for agents that need external information

Recap

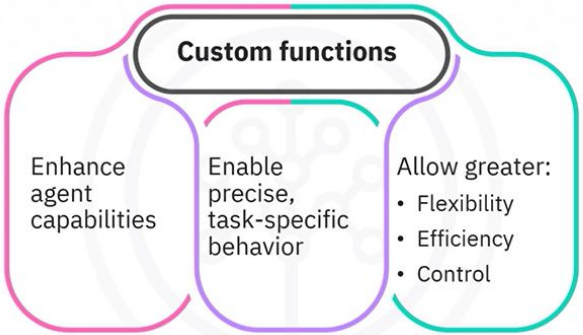
- Outputs can be stored in various formats, such as .json for shopping lists and .md for guides, depending on task requirements
- YAML allows you to define agents and tasks outside of Python, simplifying updates without touching code
- The @CrewBase decorator loads YAML-defined components as methods, making them easy to call and integrate into a Python script or notebook
- The full workflow ends with a summary agent that combines all outputs into a structured meal planning guide

What you will learn

Explain the process of creating custom functions in CrewAI

Describe how these tools can be assigned to agents and tasks

Introduction



Create a custom addition tool

- **Tools**
  - Define functional components
  - Perform specific actions
  - Support built-in and custom use

```
from crewai.tools import tool
import re
@tool("Add Two Numbers Tool")
def add_numbers(data: str) -> int:

    """
    Extracts and adds integers from the input
    string.
    Example input: 'add 1 and 2' or '[1,2,3,4]'
    Output: sum of the numbers
    """

    numbers = list(map(int, re.findall(r'-?\d+',
    data)))
    return sum(numbers)
```

Create a custom addition tool

- **@tool decorator**
  - Registers custom functions
  - Comes from crewai.tools
  - Mirrors LangChain's pattern

```
from crewai.tools import tool
import re
@tool("Add Two Numbers Tool")
def add_numbers(data: str) -> int:

    """
    Extracts and adds integers from the input
    string.
    Example input: 'add 1 and 2' or '[1,2,3,4]'
    Output: sum of the numbers
    """

    numbers = list(map(int, re.findall(r'-?\d+',
    data)))
    return sum(numbers)
```

Create a custom addition tool

- **Example: add_numbers**
  - Handles addition logic
  - Targets a calculator agent
  - Includes a descriptive docstring

```
from crewai.tools import tool
import re
@tool("Add Two Numbers Tool")
def add_numbers(data: str) -> int:
    """
    Extracts and adds integers from the input
    string.
    Example input: 'add 1 and 2' or '[1,2,3,4]'
    Output: sum of the numbers
    """
    numbers = list(map(int, re.findall(r'-?\d+',
    data)))
    return sum(numbers)
```

Create a custom multiplication tool

- Uses @tool decorator to register
- Extracts numbers from input
- Returns the product

```
from crewai.tools import tool
import re
from functools import reduce

@tool("Multiply Numbers Tool")
def multiply_numbers(data: str) -> int:
    """
    Extracts and multiplies integers from the input
string.
    Example input: 'multiply 2 and 3' or '[2,3,4]'
    Output: the product of all numbers found
    """
    numbers = list(map(int, re.findall(r'-?\d+',
data)))
    return reduce(lambda x, y: x * y, numbers, 1)
```

Define the calculator agent

- Assigns role "Calculator"
- Sets a goal to extract, add, or multiply numbers
- Highlights in the backstory its ability to interpret numeric instructions
- Assigns tools: add_numbers and multiply_numbers
- Equips the agent to handle calculation tasks

```
from crewai import Agent

calculator_agent = Agent(
    role="Calculator",
    goal="Extracts, adds, or multiplies numbers when
    asked, using the Add Two Numbers and Multiply
    Numbers tools.",
    backstory="An expert at parsing numeric
    instructions and computing sums or products.",
    tools=[add_numbers, multiply_numbers],
    llm=llm,
    allow_delegation=False
)
```

Run the crew

- Create a task
- Assign it to an agent
- Run the crew

Agent

- Extracts numbers
- Performs operation
- Returns result

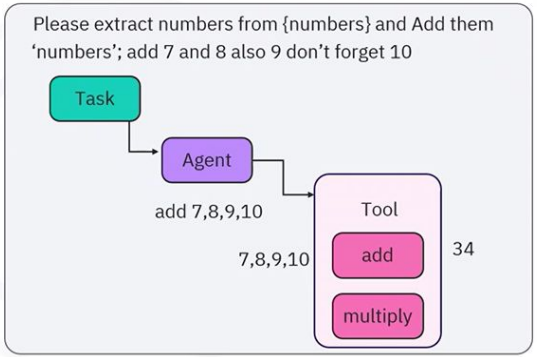
```
from crewai import Task, Crew

calculation_task = Task(
    description="Extract numbers from '{numbers}'
    and either add or multiply them, depending on
    the natural-language instruction.",
    expected_output="An integer result (sum or
    product) based on the user's request.",
    agent=calculator_agent
)
crew =
Crew(agents=[calculator_agent],tasks=[calculation_
task])
result = crew.kickoff(inputs={'numbers': 'add 7
and 8 also 9 dont forget 10'})
print("Final result:", result)
```

Process visualization

Process overview

- Two tools: add and multiply
- Input: "add 7 and 8, also 9, don't forget 10"
- Agent
  - Parses input
  - Extracts numbers
  - Selects the add tool
  - Returns output



Build a Q&A bot

- Start with the FAQ document
- Use tools to answer common questions
- Compare methods of assigning tools

The Daily Dish - Frequently Asked Questions

General Information & Location

1. Q: What are your hours of operation?
   A: The Daily Dish is open Monday to Friday from 11:00 AM to 10:00 PM, and on Saturday and Sunday from 10:00 AM to 11:00 PM.

2. Q: Where are you located?
   A: We are located at 123 Culinary Avenue, Foodie Town, FT 54321.

3. Q: What is your phone number?
   A: You can reach us at (555) 123-4567.

4. Q: Do you have parking available?
   A: Yes, we offer complimentary valet parking. There is also street parking available nearby.

Reservations & Seating

5. Q: Do I need a reservation?
   A: While walk-ins are welcome, reservations are highly recommended, especially on weekends and for larger parties. You can book a table through our website or by calling us.

6. Q: How can I make a reservation?
   A: You can make a reservation online through our website or by calling us directly at (555) 123-4567.

Powering the Q&A bot

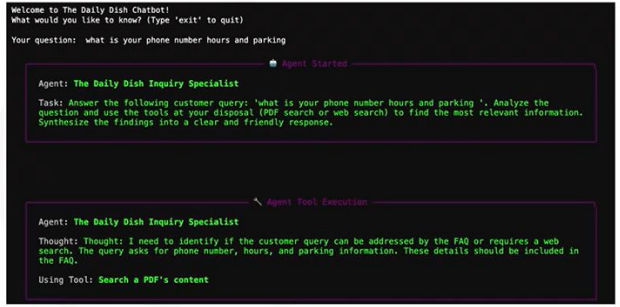
- PDFSearchTool
  - Employs the RAG tool for querying PDF files
  - Loads the FAQ PDF for program info
  - Uses sentence transformers to find relevant answers
- SerperDevTool
  - Performs real-time web search for out-of-FAQ queries

```
from crewai_tools import PDFSearchTool, SerperDevTool
pdf_search_tool = PDFSearchTool(
    pdf="/The-Daily-Dish-FAQ.pdf",
    config=dict(
        embedder=dict(
            provider="huggingface",
            config=dict(
                model="sentence-transformers/all-
MiniLM-L6-v2"
            )
        )
    )
)

web_search_tool = SerperDevTool()
```

Agent in action

- What are your phone number, hours, and parking?
  - Breaks down the question
  - Selects FAQ PDF
  - Uses PDFSearchTool
  - Finds the answer



Task-centric approach

- Define agent
- Set goal
- Explain backstory
- Assign no tools

```
    goal="Provide exceptional customer service by
    following a multi-step process to answer
    customer questions accurately.",
    backstory="""You are an AI assistant for 'The
    Daily Dish'.
    You are an expert at following instructions. You
    will be given a sequence of tasks to complete.
    For each task, you will be provided with the
    specific tool needed to accomplish it.
    Your job is to execute each task diligently and
    pass the results to the next step.""",
    tools=[], # The agent is not given any tools
directly
    verbose=True,
    allow_delegation=False,
    llm=llm
)
```

Define the task-centric agent

- Create FAQ Search task using pdf_search_tool
- Create Response Drafting task using FAQ results

```
    the information was not found.",
    tools=[pdf_search_tool],  # Tool assigned
directly to the task
    agent=task_centric_agent
)

response_drafting_task = Task(
    description="Using the information gathered from
    the FAQ search, draft a friendly and
    comprehensive response to the customer's query:
    '{customer_query}'.",
    expected_output="The final, customer-facing
response.",
    agent=task_centric_agent,
    context=[faq_search_task]
)
```

Attach tools to tasks

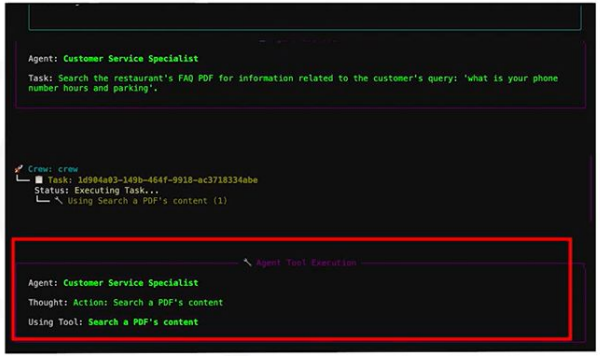
- Assign agent and tasks
- Set process to sequential
- Simulate chatbot with .kickoff()
- Demonstrate tool-task isolation and traceable steps

```
        print("Please type a question.")
        continue

    try:
        result_task_centric =
task_centric_crew.kickoff(inputs={'custom
er_query': user_input})
        print("\n--- The Daily Dish
Assistant ---")
        print(result_task_centric)
    except Exception as e:
        print(f"An error occurred: {e}")
```

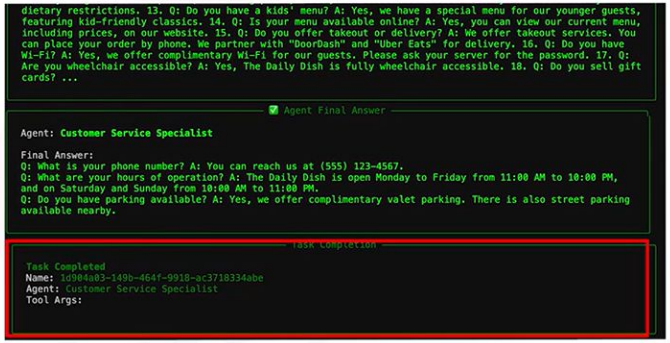
Task-centric crew execution

- Run crew with: "What is your phone number, hours, and parking?"
- Begin with the PDF search task
- Assign a Customer Service Specialist agent
- Follow instructions using PDFSearchTool



Task-centric crew execution

- Retrieve relevant information
- Assemble clear responses
- Mark task complete after formatting and return



Recap

- Custom functions enhance CrewAI by enabling domain-specific tools that improve flexibility and control.
- An agent-centric workflow assigns tools directly to the agent, letting them choose the best tool based on the query.
- A task-centric workflow attaches tools to individual tasks, guiding the agent through a fixed process.

What you will learn

- Understand AG2's core agent concepts
- Set up and configure AG2 with an LLM
- Run agents with distinct roles and human input modes
- Orchestrate agents using collaboration patterns

What is AG2 (AutoGen)?

Features

- Open-source framework for building AI agents
- Multi-agent collaboration to solve complex problems
- Provider-agnostic design that works with LLMs
- Human integration to support human-in-the-loop workflows
- Robust error handling and scalability features

Setup and installation of AG2

Installation and imports

- Import core agent types and orchestration tools

```
pip install ag2[openai]
```

```
from autogen import ConversableAgent, AssistantAgent, UserProxyAgent
from autogen import GroupChat, GroupChatManager
from autogen.llm_config import LLMConfig
```

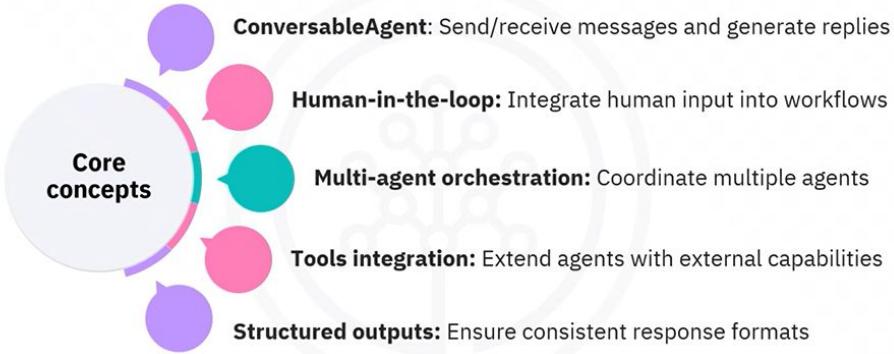
Setup and installation of AG2

LLM configuration

- Apply shared configuration or tailor per agent

```
llm_config = LLMConfig(api_type="openai", model="gpt-4o-mini")
```

AG2 agent architecture



ConversableAgent: The foundation

- Fundamental building block of AG2
- Inherited by all other agent types

ConversableAgent: Example

- Each with a distinct role defined by system messages

```
# Create specialized agents
with llm_config:
    student = ConversableAgent(
        name="student",
        system_message="You are a curious student who asks clear questions",
        human_input_mode="NEVER")

    tutor = ConversableAgent(
```

ConversableAgent: Example

- initiate_chat starts the conversation

```
# Start a conversation
chat_result = student.initiate_chat(
    recipient=tutor,
    message="Can you explain what a neural network is?",
    max_turns=2,
    summary_method="reflection_with_llm")
```

Benefits of Multi-Agent Approach

- Yields better results
- Simulates natural, interactive learning
- Structured reasoning through role-based collaboration
- Higher quality outputs from specialized perspectives
- Context-rich answers from distributed reasoning
- Scalable to complex problems

Built-in agent types

AssistantAgent

- Handles problem-solving
- Generates code

UserProxyAgent

- Handles code execution
- Provides feedback

Code generation and execution example

```
# Create code-writing assistant
assistant = AssistantAgent(
    name="assistant",
    system_message="Helpful assistant who writes clear Python code")

# Create code-executing user proxy
user_proxy = UserProxyAgent(
```

Code generation and execution example

- LocalCommandLineCodeExecutor provides safe execution environment

```
# Create code-executing user proxy
user_proxy = UserProxyAgent(
    name="user_proxy",
    human_input_mode="NEVER",
    max_consecutive_auto_reply=5,
    code_execution_config={
        "executor": LocalCommandLineCodeExecutor(work_dir="coding")})
```

Code generation and execution example

Caution:
Please exercise discretion when using the code execution functionality.

Code generation and execution example: Output

- User proxy executes code automatically

```
user_proxy (to assistant): Plot a sine wave using matplotlib from -2π to 2π
and save the plot as sine_wave.png.

--------------------------------------------------
assistant (to user_proxy): Sure! To plot a sine wave using Matplotlib and save
the plot as `sine_wave.png`, you can use the following Python code:

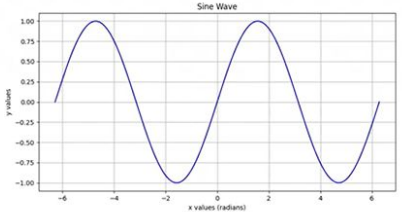

```python import numpy as np


import matplotlib.pyplot as plt

# Generate x values
```

Code generation and execution example: Output

Plot features labeled axes and blue line for $y = sin(x)$

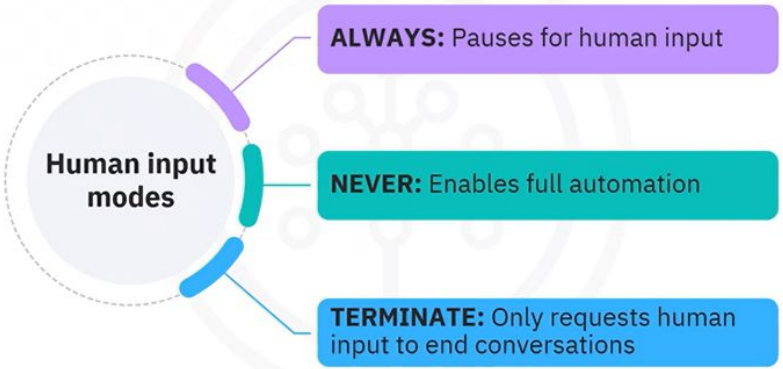


Final summary confirms image was saved

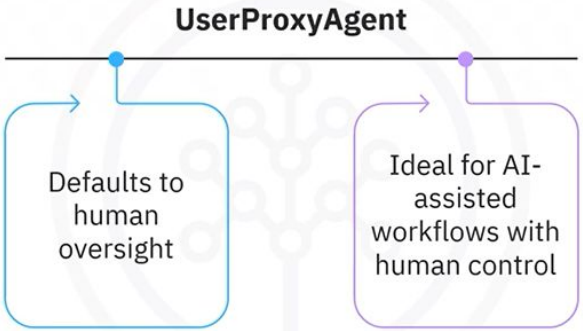
Final Summary: The sine wave was successfully plotted using Matplotlib, and the output image `sine_wave.png` was generated without errors. The user can check the working directory for the saved image and view the plot. Suggestions for modifications to enhance the plot were provided, including changing colors, styles, and formats for saving. The user can seek further assistance for specific changes or questions about data visualization in Python.

Human-in-the-loop

Essential for finance, healthcare, and law



Human-in-the-loop


```

Human-in-the-loop: Example

- Escalates critical bugs or closes minor ones

```
# Bug triage system with human oversight
triage_system_message = """
You are a bug triage assistant. For each bug report:
- Urgent issues (crash, security, data loss): escalate and ask for
  confirmation
- Minor issues (cosmetic, typos): suggest closing but ask for review
- Otherwise: classify as medium priority and ask for review
"""
```

Multi-agent orchestration

Why Multi-agent systems?

Assign specific roles

Incorporate human oversight at key decision points

Enable agents to build on each other's outputs using:

- Two-agent chat: Simple task exchanges
- Group chat: Diverse, multi-purpose discussions
- Sequential chat: Step-by-step refinement
- Nested chat: Reusable sub-conversations

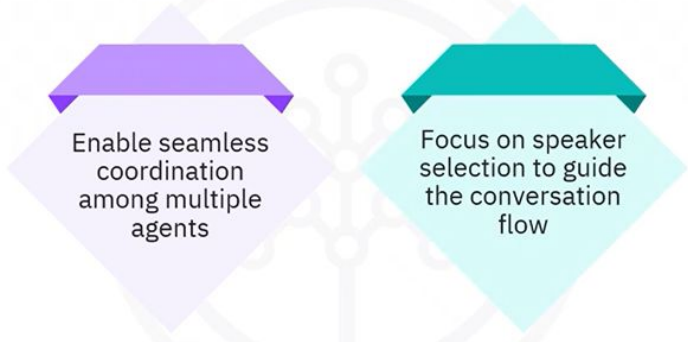
Keep complex systems organized, scalable, and manageable

GroupChat and GroupChatManager

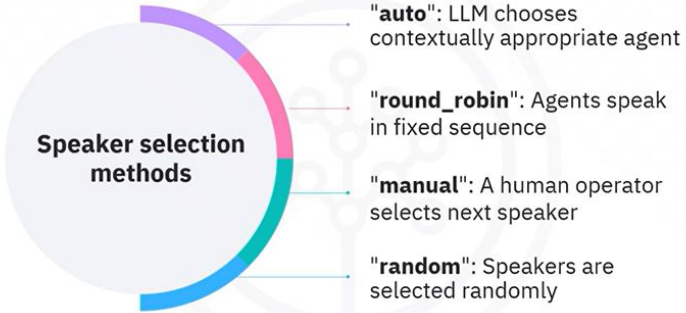
```
## GroupChat Components:
groupchat = GroupChat(
    agents=[teacher, planner, reviewer],
    speaker_selection_method="auto"  # LLM selects next speaker)

manager = GroupChatManager(
    name="group_manager",
```

GroupChat and GroupChatManager



GroupChat and GroupChatManager



GroupChat: Lesson planning workflow example

Lesson planner: Creates educational content

```
# Create specialized education agents
lesson_planner = ConversableAgent(
    name="planner_agent",
    system_message="Create lesson plans for 4th graders",
    description="Makes lesson plans")

lesson_reviewer = ConversableAgent(
```

GroupChat: Lesson planning workflow example

Reviewer: Provides feedback

```
lesson_reviewer = ConversableAgent(
    name="reviewer_agent",
    system_message="Review plans and suggest up to 3 brief edits",
    description="Reviews lesson plans and suggests edits")

teacher = ConversableAgent(
    name="teacher_agent",
```

GroupChat: Lesson planning workflow example

Teacher: Oversees the process

```
    name="reviewer_agent",
    system_message="Review plans and suggest up to 3 brief edits",
    description="Reviews lesson plans and suggests edits")

teacher = ConversableAgent(
    name="teacher_agent",
    system_message="Suggest topics and reply DONE when satisfied",
    is_termination_msg=lambda x: "DONE" in x.get("content", "").upper())
```

Group chat coordination

- Use GroupChatManager to monitor the interaction

```
manager = GroupChatManager(
    name="group_manager",
    groupchat=groupchat,
    llm_config=llm_config)

# Start with a short initial prompt to keep tokens low
```

Group chat coordination

- Use GroupChatManager to monitor the interaction

```
# Start with a short initial prompt to keep tokens low
teacher.initiate_chat(
    recipient=manager,
    message="Make a simple lesson about the moon.",
    max_turns=6, # Limit total rounds (e.g., 2 per agent max)
    summary_method="reflection_with_llm")
```

Recap

- AG2 is an open-source framework for building intelligent AI agents that collaborate through structured, role-based interactions
- AG2's core concepts, like ConversableAgent, human-in-the-loop, tools integration, and multi-agent orchestration, enable flexible, role-based collaboration
- Agents are configured with LLM settings and system messages to define behavior, roles, and communication flows for specialized tasks
- Human-in-the-loop modes provide oversight options ranging from full automation to manual approval at key checkpoints
- Multi-agent orchestration patterns, such as group chat and sequential flows, coordinate multiple agents to solve complex problems interactively

What you will learn

Extend AG2 agents with tools for actions such as code execution and API calls

Use structured outputs with Pydantic models for consistent, predictable responses

Apply best practices for secure, reliable AG2 production setups

AG2 agent architecture

Core concepts

- Conversable agent
- Human-in-the-loop
- Multi-agent orchestration
- **Tools**
- **Structured outputs**

Tool integration support

Why tools matter?

Extend agent capabilities beyond language generation

Enable actions like code execution, calculations, and output generation

Connect to APIs or databases

- Access real-time data
- Interact with file systems
- Integrate with web services

Support automation, analysis, and visualization

Make agents more effective in real-world workflows

Tool registration

- is_prime function checks if a number is prime
- register_function links the tool between the agents
- math_asker requests the check
- math_checker runs the function

```
def is_prime(n: int) -> str:
    """Check if a number is prime"""
    if n < 2: return "No"
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0: return "No"
    return "Yes"

register_function(
    is_prime,

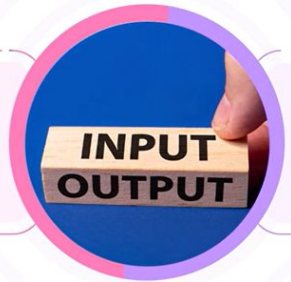
    caller=math_asker, #Agent that requests the tool
    executor=math_checker, #Agent that executes the tool

    description="Check if a number is prime. Returns Yes or No.")
```

Structured outputs

A world without consistent output

Agent responses can vary

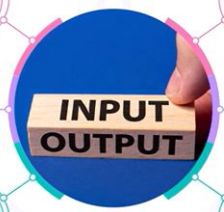


Agents can be unreliable for processing or API integration

Structured outputs

How do structured outputs help?

- Enforce a clear, consistent format
- Validate data to maintain format consistency
- Support reliable API integration
- Prevent malformed responses
- Enforce type safety
- Make the system robust, predictable, and production-ready



Structured outputs: Implementation with Pydantic example

- Create a TicketSummary model using Pydantic
- Set the model as response_format in llm_config
- Make it structured, reliable, and ready for integration

```
class TicketSummary(BaseModel):

    customer_name: str
    issue_type: str
    urgency_level: str
    recommended_action: str
```

```
llm_config = LLMConfig(
    api_type="openai",
    model="gpt-4o-mini",
    response_format= TicketSummary # Enforces structure)
```

Best practices: Security and configuration

- Avoid hardcoding API keys
- Use environment variables or secure credential stores
- Use config lists and fallback models to maintain uptime
- Adjust temperature settings
  - 0.0 for consistent, structured outputs
  - 0.7–1.0 for creative work
- Implement rate limiting and robust error handling for stability at scale

Best practices: Agent design

- Craft strong system messages
- Set max_consecutive_auto_reply to prevent infinite loops
- Choose the right human_input_mode
- Keep agents specialized with well-defined tasks

Best practices: Agent design

Clear boundaries lead to more reliable, maintainable behavior

Best practices: Human-in-the-loop implementation

- Use human-in-the-loop where human intervention adds value
- Define clear escalation criteria (risk levels, financial thresholds)
- Provide sufficient context to human reviewers
- Log all human interventions

Best practices: Multi-agent orchestration

- Design clear workflows with defined handoffs between agents
- Use GroupChat for collaborative problem-solving
- Implement termination conditions to prevent endless conversations
- Monitor conversation quality and human intervention points

Best practices: Tools integration

- Create specific tools for focused purpose
- Implement proper error handling to manage potential failures
- Validate tool inputs and outputs thoroughly
- Document tool capabilities clearly for agents

Best practices: Structured outputs

For effective structured outputs

Recap

- AG2 agents can be extended with tools to perform real-world actions such as code execution, API integration, and data processing, allowing them to automate tasks, access live data, and execute complex workflows.
- Structured outputs with Pydantic models enforce consistent response formats, validate data automatically, and prevent errors during API or automated workflow integration.
- Best practices for AG2 production setups include securing credentials, using fallback models for reliability, tuning temperature settings for the task, and adding robust error handling for stability.
- Clear agent roles and orchestration patterns, combined with targeted human-in-the-loop strategies, enhance the accuracy, efficiency, and maintainability of multi-agent systems.